

MCT-454L Mobile Robotics – Lab 2 Report

Topic: ROS 2, Packages, Nodes, Topics, Services, and RQT

Objective

- Familiarize with ROS 2 command-line interface (CLI) tools.
- Understand the basic operation of a simulated robot (Turtlesim).
- Learn how to control a robot using ROS 2 topics.
- Learn how to interact with ROS 2 services to perform actions like reset, spawn, or teleport.
- Explore the ROS 2 ecosystem visually using the `rqt` GUI.
- Observe the real-time effect of commands and service calls on robot behavior.

Steps Followed

Before starting the lab, I wanted to make sure that ROS 2 and my workspace were always ready. Manually sourcing setup files every time can be tedious and easy to forget. To fix this, I made the environment permanent by adding the following lines to my `.bashrc` file:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

With this, every new terminal automatically recognized ROS 2 commands and workspace packages. No repeated steps. Everything was ready from the start, which made the workflow smooth and stress-free.

Next, I checked whether the `turtlesim` package was installed. It wasn't, so I installed it using:

```
sudo apt install ros-humble-turtlesim
```

Step 1: Launching Turtlesim

I started the Turtlesim node with:

```
ros2 run turtlesim turtlesim_node
```

A small turtle appeared on a blue canvas. Watching it idle was surprisingly engaging. Even before giving any commands, the simulation already felt alive.

Step 2: Controlling the Turtle

In a new terminal, I launched the teleop node:

```
ros2 run turtlesim turtle_teleop_key
```

Using the keyboard, I could move the turtle forward, backward, and rotate it. The turtle responded instantly to every command. It felt natural and intuitive.

Step 3: Exploring ROS 2 Topics

First, I listed all active topics:

```
ros2 topic list
```

This showed topics like `/turtle1/cmd_vel`, `/turtle1/color_sensor`, and `/turtle1/pose`.

Next, I inspected specific topics to understand their structure:

```
ros2 topic info /turtle1/cmd_vel
ros2 topic info /turtle1/color_sensor
ros2 topic info /turtle1/pose
```

For `/turtle1/cmd_vel`, the output revealed:

- **Type:** `geometry_msgs/msg/Twist`
- **Publisher count:** 1
- **Subscription count:** 1

This told me that the turtle was subscribed to receive velocity commands. The command I publish would move it immediately.

For `/turtle1/color_sensor` and `/turtle1/pose`, I could see how sensor and position data were communicated in real-time. Exploring these topics gave me a clear picture of ROS 2's publisher-subscriber model.

I also experimented with:

```
ros2 topic echo /turtle1/cmd_vel
```

This displayed the live velocity commands being sent to the turtle. It was satisfying to see numbers on the screen translate directly into motion.

Step 4: Sending Velocity Commands

I sent a command to move the turtle:

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: 1.8}}"
```

The turtle moved smoothly and spun as expected. Watching it respond instantly felt almost magical.

Step 5: Resetting the Simulation

Finally, I reset the turtle's position using:

```
ros2 service call /reset std_srvs/srv/Empty
```

The turtle returned to its starting point immediately. This clearly demonstrated how services control the simulation in real-time.

Exercise: Getting Familiar with rqt and Calling Services

I began by opening **rqt** in a terminal. Instantly, a graphical interface appeared, showing the ROS 2 ecosystem in a neat visual layout. Using the **Plugins menu**, I explored nodes, topics, and services. The **Node Graph** plugin was especially useful, it displayed all active nodes and how they connect through topics. Seeing this network made it much easier to understand message flow in ROS 2.

Next, I navigated to the **Services tab**. I first called the **/reset** service. Instantly, the turtle jumped back to its starting position. It was satisfying to see a single command have such an immediate effect. Then, I used the **/spawn** service to create a second turtle. I set the x, y, theta, and gave it a name. As soon as I called the service, the new turtle appeared, ready to move. Watching the simulation expand dynamically was exciting.

To control the second turtle, I identified the topic **/turtle2/cmd_vel**. I sent velocity commands from the terminal using:

```
ros2 topic pub -r 10 /turtle2/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: 1.5}}"
```

Both turtles moved independently. The second turtle responded to my commands while the first continued on its own path. This demonstrated how ROS 2 can manage multiple nodes and topics simultaneously.

I also changed the background color using terminal commands:

```
ros2 param set /turtlesim background_r 255
```

```
ros2 param set /turtlesim background_g 0
```

```
ros2 param set /turtlesim background_b 0
```

ros2 service call /clear std_srvs/srv/Empty

The background updated instantly. Seeing colors change in real time made the simulation more interactive and visually appealing.

Finally, I experimented with **teleporting turtles using services**, both **absolutely and relatively**.

- **Absolute teleportation** moves the turtle to a specific x, y, and theta coordinate, regardless of its current position.
- **Relative teleportation** moves the turtle by a certain distance and angle relative to its current position.

Each teleportation call was reflected immediately in the simulation.

Conclusion

This lab was a hands-on dive into ROS 2. I got to see nodes, topics, and services working together in real time. Controlling the turtles, exploring topics, and calling services through both the terminal and rqt made ROS 2's flexibility very clear. Watching multiple turtles move independently, resetting simulations, and experimenting with teleportation and background changes made the session interactive.